

EPFL CIVIL-127, Lab 5

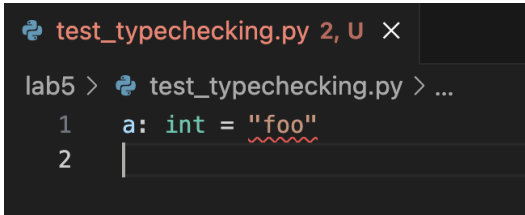
March 17, 2026

This week, continue working in the same **CIVIL-127** folder as previous weeks. Create a subfolder called **lab5**.

It is recommended that you enable type checking. You can use the settings menu in VSCode or create a subfolder called **.vscode**. Inside this folder, create a file called **settings.json** with the following content:

```
{
  "python.analysis.typeCheckingMode": "strict"
}
```

Check that type checking is working by writing the following code. Do you get red squiggles?



```
test_typechecking.py 2, U X
lab5 > test_typechecking.py > ...
1   a: int = "foo"
2   |
```

Install pygame:

- From within VSCode, open a Terminal
 - Your terminal prompt should look like **(civil127) username@hostname civil127 %**. If it doesn't, your venv might not be configured properly. Check that you have a **.venv** subfolder (you should have had one since week 1)
 - Run the following command. It will create a couple files, you don't need to touch any of these files.
uv init
 - Install pygame with:
uv add pygame-ce==2.5.6

Make sure your installation was successful. Create a file **lab5/check_pygame.py** with the following content and run the code. Do you see a window with a blue rectangle on white background?

```
import pygame

def main() -> None:
    pygame.init()
    window = pygame.Window()
    clock = pygame.time.Clock()
    while True:
        for event in pygame.event.get():
            print(event)
        window.get_surface().fill(pygame.Color("white"))
        pygame.draw.rect(
            window.get_surface(),
            pygame.Color("blue"),
            (10, 10, 50, 20))
        window.flip()
        clock.tick(60)

main()
```

Create a file `lab5/a.py` with the content:

```
def in_a():
    print("in a")
```

Create a file `lab5/b.py` with the content:

```
import lab5.a as a

print("in b")
a.in_a()
```

To run this code, you'll need to configure `.vscode/launch.json`. In VSCode, go to "run > Add Configuration...", pick "Python Debugger" followed by "Module" and enter "lab5.b". Your launch.json should look like:

```
{
  // Use IntelliSense to learn about possible attributes.
  // Hover to view descriptions of existing attributes.
  // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug b.py",
      "type": "debugpy",
      "request": "launch",
      "module": "lab5.b"
    }
  ]
}
```

Make sure you can run `b.py`:

```
in b
in a
```

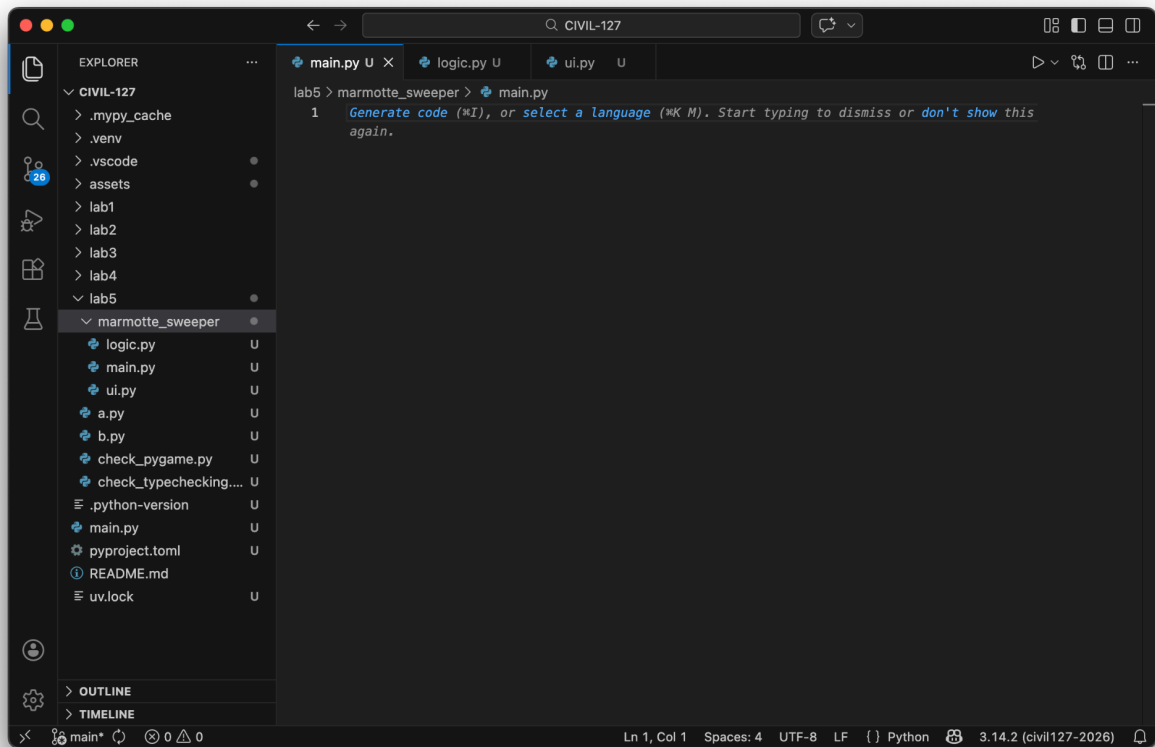
5.1

You will be implementing the first part of Marmotte Sweeper. The goal for this week is to draw the grid in a window and allow the user to flag/unflag cells (as explained in class).

Download [assets.zip](#) and unzip the images in `CIVIL-127/assets`. Create a subfolder `CIVIL-127/lab5/marmotte_sweeper` and put 3 empty files in there: `main.py`, `logic.py`, and `ui.py`. You are going to split the game code across `logic.py` and `ui.py`:

- `logic.py` will contain code that tracks where each marmotte is located, which cells have been revealed, and which cells have been flagged. Here, all the coordinates are grid-based. This code must not know anything about being a graphical application.
- `ui.py` will contain code that creates the window, draws the cells, and transfers mouse click events to `logic.py`. Here, the coordinates are pixels and get converted to/from grid coordinates as needed.

Your `CIVIL-127/` folder should look like:



In `lab5/marmotte_sweeper/logic.py`, implement the following. This should take ~80 lines of code.

- A class `Cell` which extends `Enum` and takes 4 values: `UNREVEALED`, `MARMOTTE`, `REVEALED`, `FLAGGED`.
- `type coord = tuple[int, int]`
- A class `MarmotteSweeper`
 - Implement the `__init__` method, which takes `grid_width`, `grid_height` and `count_marmottes` as parameters
 - Store each parameter since you might need them later (i.e. `self.grid_width=grid_width`, `self.grid_height=grid_height`, etc.)
 - Initialize a set, `self.marmottes`, of type `set[coord]` to track the positions of the marmottes. Place `count_marmottes` by randomly picking coordinates in the `0` to `grid_width`, `0` to `grid_height` range.
 - Initialize an empty set, `self.revealed`, of type `set[coord]` to track which positions have been revealed.
 - Initialize an empty set, `self.flagged`, of type `set[coord]` to track which positions have been flagged.
 - Implement a `flag` method that takes `pos` of type `coord` as a parameter
 - If `pos` exists in `self.flagged`, remove it
 - Otherwise, add `pos` to `self.flagged`

- Implement a reveal method that takes `pos` of type `coord` as a parameter
 - Add `pos` to `self.revealed`
- Implement a `cell` method that takes `pos` of type `coord` as a parameter and returns a `Cell`
 - If `pos` is in `self.revealed` and in `self.marmottes`, return `Cell.MARMOTTE`
 - Otherwise if `pos` is in `self.revealed`, return `Cell.REVEALED`
 - If `pos` is in `self.flagged`, return `Cell.FLAGGED`
 - Otherwise, return `Cell.UNREVEALED` which is the default/initials state for each cell

In `lab5/marmotte_sweeper/ui.py`, implement the following. This should also take ~80 lines of code.

- A class `MarmotteSweeperUI`
 - Implement the `__init__` method, which takes a `grid_width`, `grid_height`, `count_marmottes`, `cell_size`
 - Store each parameter since you might need them later.
 - Initialize


```
self.marmotte_sweeper=MarmotteSweeper(grid_width,
grid_height, count_marmottes)
```
 - Load `blank.png` and `flag.png`. Scale the images to `cell_size` x `cell_size`. If you want, you can load all the other images too and think about what data structure you want to put them in.
 - Call `pygame.init()`
 - Create a window of size `grid_width * cell_size` x `grid_height * cell_size`
 - Implement a draw method
 - Iterate over every grid position, `(x, y)` from 0 to `grid_width` and 0 to `grid_height`, call `self.marmotte_sweeper.cell((x, y))` to get the cell type and draw the corresponding image using `surface.blit(image, (x * self.cell_size, y * cell_size))` (you are going from grid coordinate to pixels here)
 - Implement a start method
 - Implement the pygame event loop
 - Handle `pygame.QUIT` event
 - Handle `pygame.MOUSEBUTTONDOWN` event
 - `event.pos` will give you the mouse position in screen coordinates, you can do `// self.cell_size` to convert them to grid coordinates
 - call `self.marmotte_sweeper.flag()` to toggle the flag

`lab5/marmotte_sweeper/main.py` creates `MarmotteSweeperUI` and calls `start()`:

```
import random

import lab5.marmotte_sweeper.ui as ui

# Code to create an instance of MarmotteSweeperUI and start
# the game.
# Adjust difficulty by changing the grid size and number of marmottes.
# Adjust the cell_size depending on the screen resolution.

random.seed(2) # for testing purpose
game = ui.MarmotteSweeperUI(9, 9, 15, 64)
game.start()
```

Add a configuration to debug `lab5.marmotte_sweeper.main`. You should now have a half working game.

5.2 (optional)

Implement unittests for `lab5.marmotte_sweeper.logic.MarmotteSweeper`